

Datasets & Data Access

Where can we get the data, and what do we do with it?

Where does the data come from?

The Magnetosphere

In-situ spacecraft measurements from the Magnetosphere:

- Geotail, Van Allen Probes, **Cluster**, THEMIS, **MMS**, etc.
- Different positions
- Different instruments

Data source matters:

- Sampling bias
 - Space
 - Time
- Data resolution

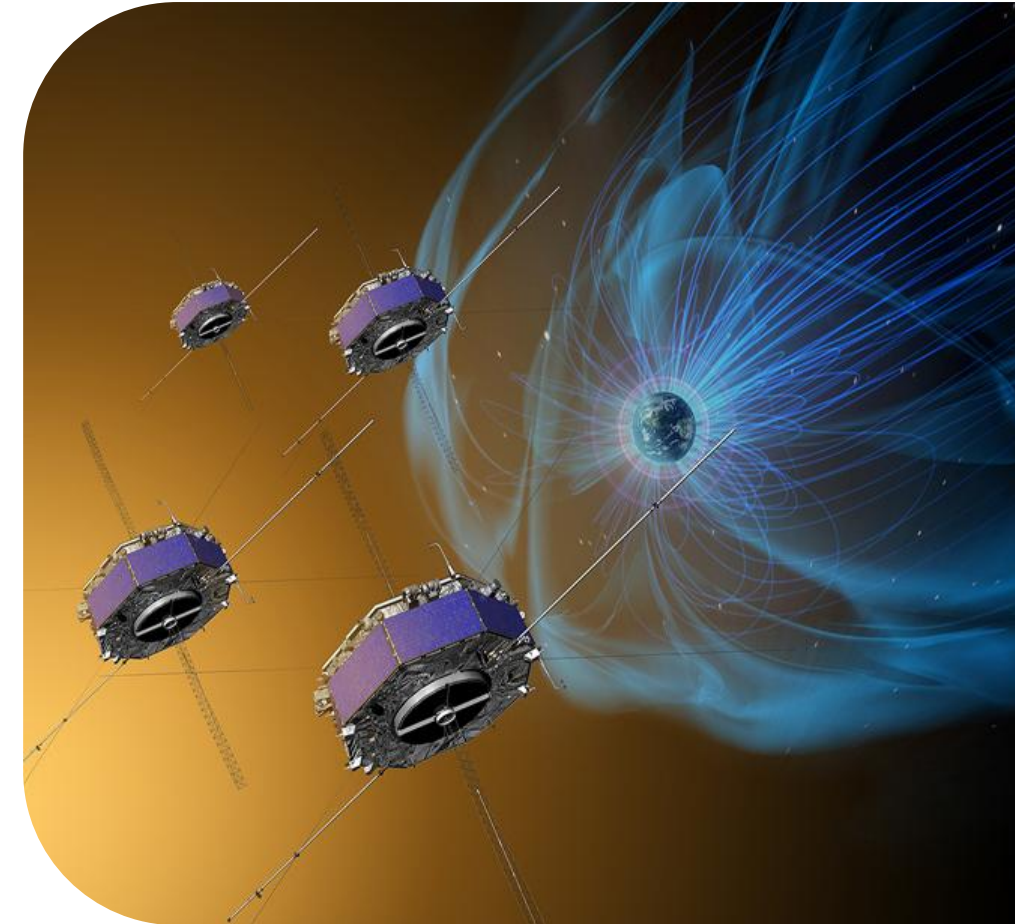


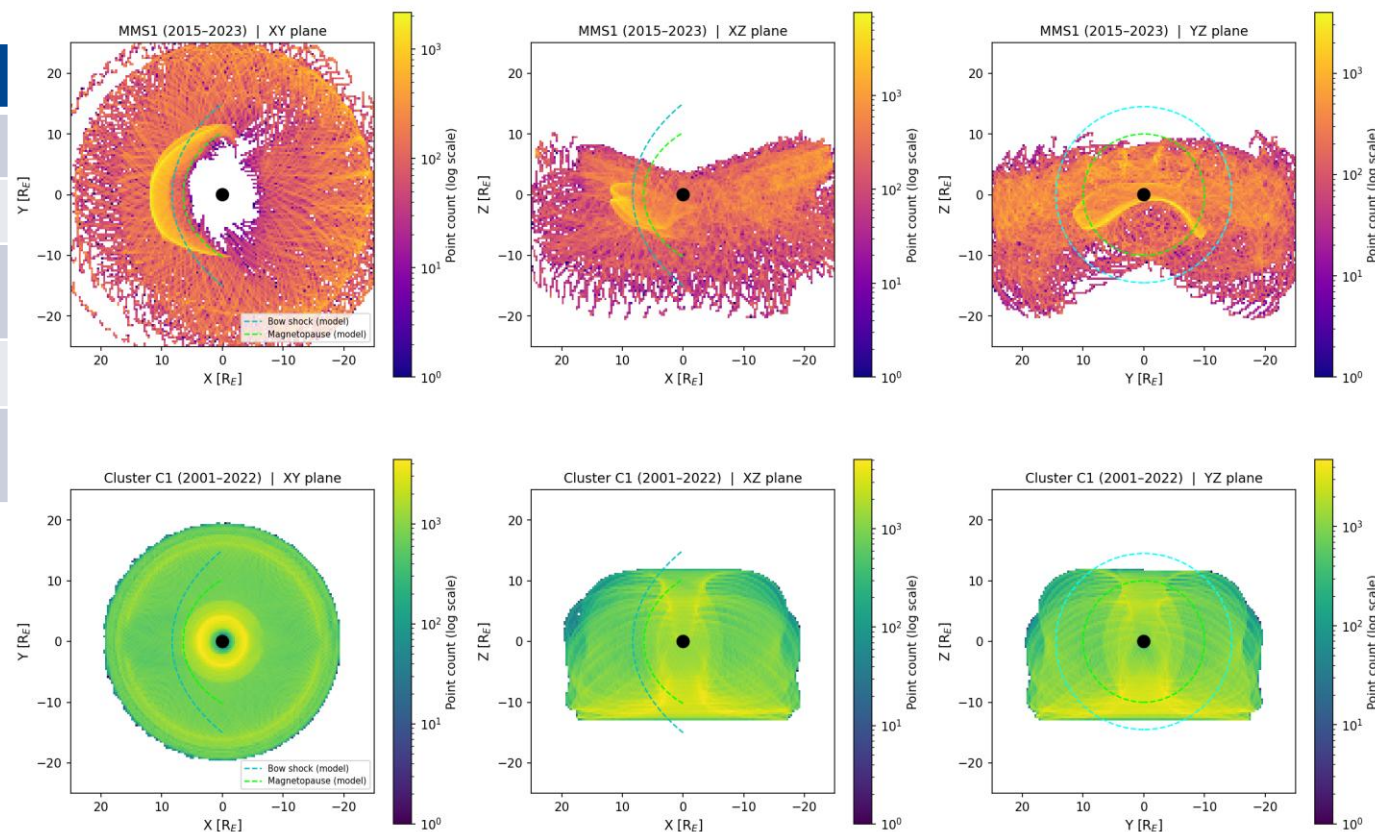
Image Credit: NASA

Where does the data come from?

Cluster & MMS

	Cluster, 4 s/c	MMS 4 s/c
Mission time	2000 - 2024	2015 -
Orbit (RE)	~4-20	~1-12 (25 in phase 2)
Target	Magnetosphere, cusp, tail	Magnetic reconnection
Ion Dist. Resolution	4s	4.5s (150ms in burst)
Strength	Long baseline	High resolution (Not all data)

MMS and Cluster — Science Data Coverage Maps (GSE, point density)



When does the data come from?

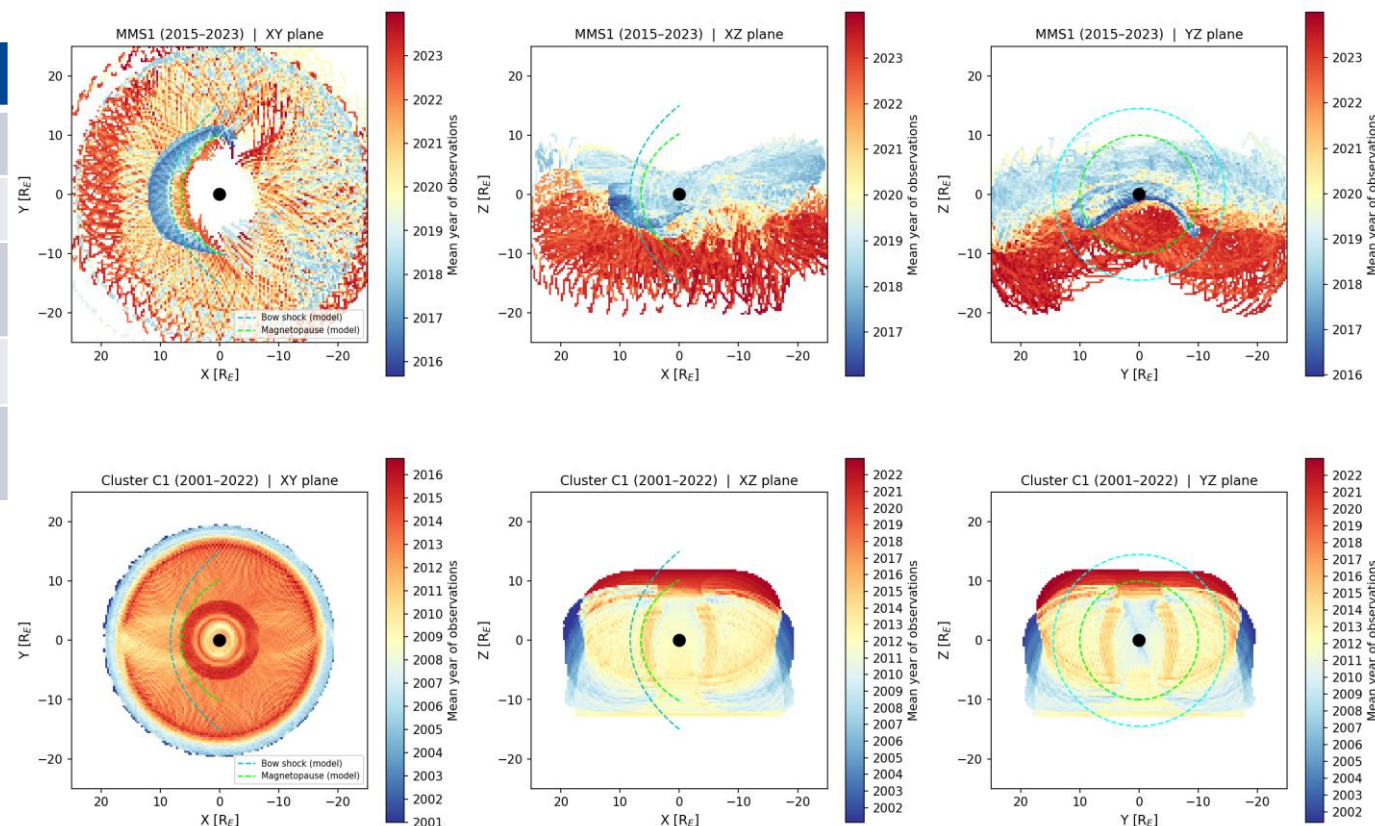
Cluster & MMS

	Cluster, 4 s/c	MMS 4 s/c
Mission time	2000 - 2024	2015 -
Orbit (RE)	~4-20	~1-12 (25 in phase 2)
Target	Magnetosphere, cusp, tail	Magnetic reconnection
Ion Dist. Resolution	4s	4.5s (150ms in burst)
Strength	Long baseline	High resolution (Not all data)

MMS Mission Phases:

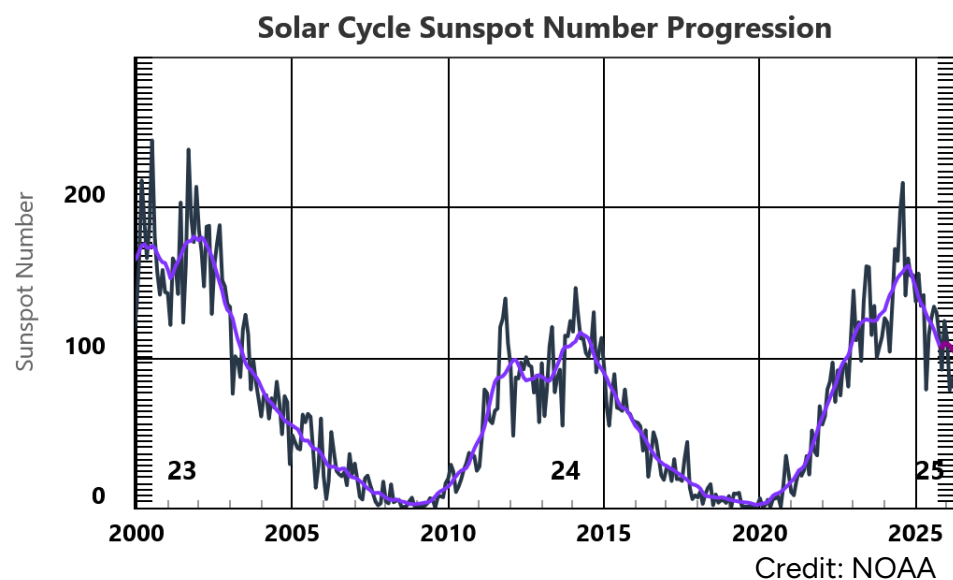
- Phase 1: 2015-2017
- Phase 2: 2017
- Extended Mission:
 - 2017->Now?

MMS and Cluster — When Was Each Region Sampled? (mean year per bin)

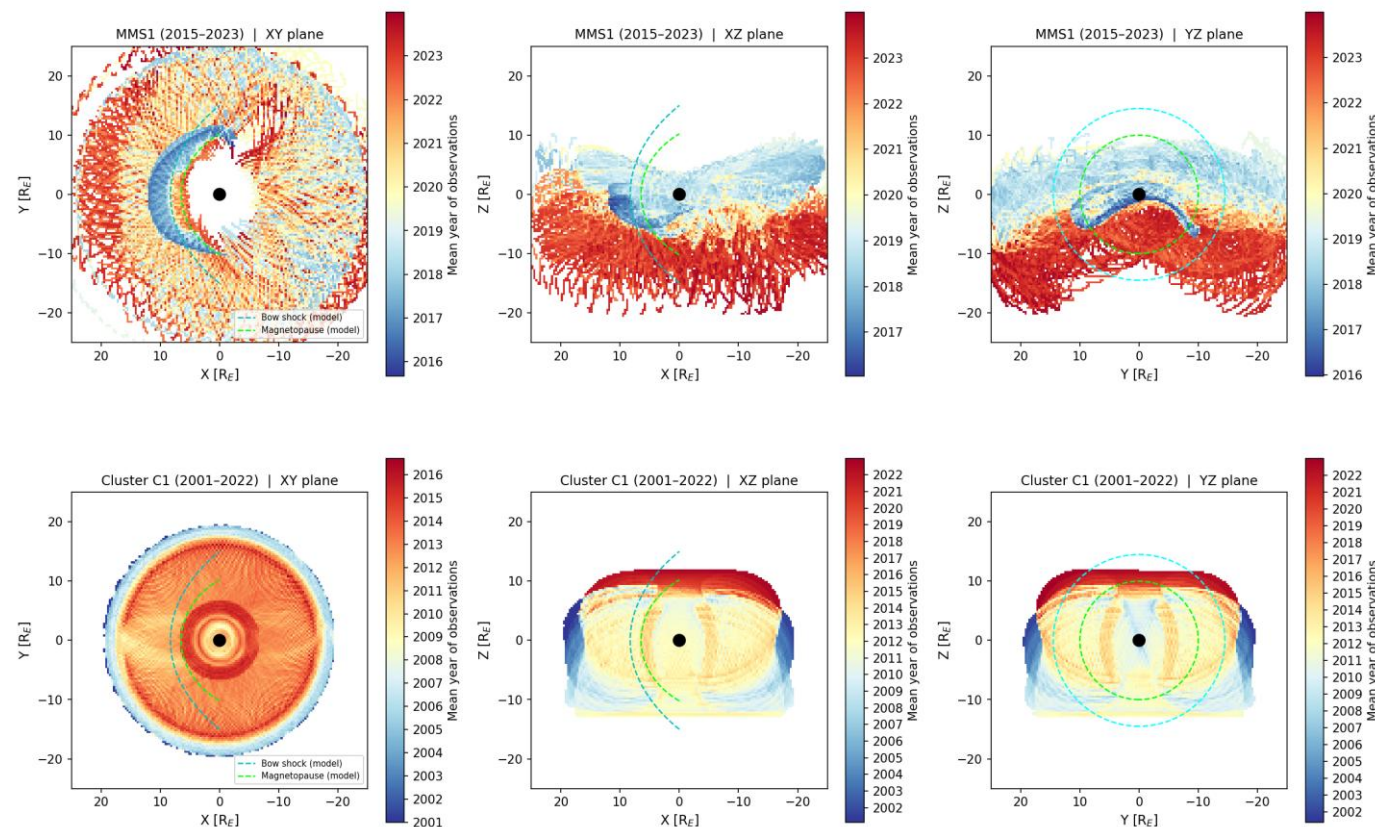


When does the data come from?

Cluster & MMS



MMS and Cluster — When Was Each Region Sampled? (mean year per bin)



Data access in practice: From remote to local

Route 0: Web Access

- **Mission webpages, CDAWeb** (cdaweb.gsfc.nasa.gov)
 - Good for quick looks at data, pre-generated plots

Route 1: Script access raw data (python)

- cdasws: Barebone Python interface to NASA's Coordinated Data Analysis System.
 - No automatic data storing, you get the data, then it is up to you.
 - Support data access from multiple missions
- Pyspedas:
 - Made for data analysis,
 - Support Data access from multiple missions.
 - Problematic for large files and long time ranges
- Speasy: <https://speasy.readthedocs.io/en/latest/>

```
import pyspedas
from pyspedas.projects import mms
from pyspedas.tplot_tools import get_data
# -- Load data --
trange = ['2017-07-11/22:00:00',
          '2017-07-11/23:00:00']

mms.fgm(trange=trange, probe='1',
        data_rate='srvy', time_clip=True)
mms.fpi(trange=trange, probe='1',
        datatype='dis-moms', data_rate='fast',
        time_clip=True)

# -- Extract numpy arrays --
t_b, B = get_data('mms1_fgm_b_gsm_srvy_l2')
t_n, ni = get_data('mms1_dis_numberdensity_fast')
```

Data access in practice: From remote to local

Route 1: Script access (python)

- SpacePhyML:
 - <https://github.com/asap-space/SpacePhyML>
 - Python Library for accessing Space Physics ML datasets and Models
 - Developed as part of ASAP
 - Support downloading data from MMS Science Data Center



```
import xarray as xr
from spacephyml.datasets.creator import create_dataset

raw_data_file = 'data/data_raw.nc'
var_list = ['mms1_dis_energyspectr_omni_fast']
trange = ["2017-11-01", "2017-12-31"]
create_dataset(raw_data_file,
               trange,
               var_list=var_list,
               label_source='Olshevsky',
               common_file=True
               )

ds = xr.open_dataset(raw_data_file)
```

ML ready datasets

- SpacePhyML
- Zenodo
- Kaggle
- Indiv. Publications.

Its limitations: Curated datasets encode choices. What was included, what was excluded, how labels were assigned, all of this shapes what a model trained on SpacePhyML will and will not learn.

```
import torch
from torch.utils.data import DataLoader
from spacephyml.datasets.mms import MMS1IonDistLabeled
from spacephyml.models.mms import PCReduced

device = ('cuda' if torch.cuda.is_available()
          else 'cpu')

dataset = MMS1IonDistLabeled('SCNov2017')
model = PCReduced('s42').to(device)

labels = {'human': [], 'classifier': []}
correct = 0
with torch.no_grad():
    for x, l in DataLoader(dataset, batch_size=32):
        lc = model(x.to(device)).to('cpu')
        lc = torch.argmax(lc, axis=1)
        correct += torch.sum(lc == l)
        labels['human'].extend(l)
        labels['classifier'].extend(lc)
print(f'Accuracy: {correct/len(labels["human"])}')
```

These are tools for accessing existing datasets, labels, and models. Read the original papers to understand the data, datasets and label choices.

Creating our dataset

- SpacePhyML:
 - Omni-directional ion spectrum
 - November and December 2017
 - Labels from Olshevsky et. al (2021).

```
raw_data_file = 'data/data_raw.nc'

var_list = ['mms1_dis_energyspectr_omni_fast']
trange = ["2017-11-01", "2017-12-31"]
create_dataset(
    str(raw_data_file),
    trange,
    var_list=var_list,
    label_source='Olshevsky',
    common_file=True,
)

ds = xr.open_dataset(raw_data_file)
```

Inspecting the data

What to ask:

- Do we have missing data?
 - Timesteps with all features missing?
 - Or a subset of features available?
- How is the data distribution?
- If we have a time axis, is it aligned?
- What is the shape of our data?
 - 1D, 2D, multiple features, time steps
- Where is our data located (time and space)

Inspecting the data

Load your data as either pandas (Table data) or xarray (Multidimensional data).

- Do we have missing data?
 - Timesteps with all features missing?
 - Or a subset of features available?

```
Fill values: 0

NaN fraction per variable:
label          0.0%
MMS1 FPI DIS energySpectr_omni 0.0%
MMS1 FPI DIS energy          0.0%
label str          [skipped - dtype: object]

Time axis per variable:
label          n=463,048 cadence=4.5s 2017-11-01T03:24:55 → 2017-12-30T19:01:17
MMS1 FPI DIS energySpectr_omni n=463,048 cadence=4.5s 2017-11-01T03:24:55 → 2017-12-30T19:01:17
MMS1 FPI DIS energy          n=463,048 cadence=4.5s 2017-11-01T03:24:55 → 2017-12-30T19:01:17
label str          n=463,048 cadence=4.5s 2017-11-01T03:24:55 → 2017-12-30T19:01:17

Gaps detected: 59
```

```
# — 1. Check for fill values —————
fillval = 1e-31
print(f"Fill values: {int(ds[\"MMS1 FPI DIS energySpectr_omni\"].isnull().sum())}\n")
ds = ds.where(ds != fillval)

# — 2. NaN CHECK —————
print("NaN fraction per variable:")
for var in ds.data_vars:
    arr = ds[var].values
    try:
        frac = float(np.isnan(arr.astype(float)).mean())
        print(f" {var:<30} {frac:.1%}")
    except (ValueError, TypeError):
        print(f" {var:<30} [skipped - dtype: {arr.dtype}]")

# — 3. TIME ALIGNMENT —————
# Check that all variables share the same time axis (cadence + extent)
print("\nTime axis per variable:")
for var in ds.data_vars:
    t = ds[var].time.values if "time" in ds[var].coords else ds.time.values
    dt = pd.to_timedelta(np.median(np.diff(t.astype("datetime64[ns]").astype(np.int64))))
    print(f" {var:<30} n={len(t):>7,} cadence={dt.total_seconds():.02}s "
          f"{str(t[0])[:19]} → {str(t[-1])[:19]}")

# — 4. TIME SERIES GAPS —————
# A gap is any step larger than 5× the median cadence
time_ns = ds.time.values.astype("datetime64[ns]").astype(np.int64)
dt_steps = np.diff(time_ns)
nominal = np.median(dt_steps)
gap_mask = dt_steps > 5 * nominal
gap_starts = ds.time.values[:-1][gap_mask]
gap_ends = ds.time.values[1:][gap_mask]
gap_dur = pd.to_timedelta(dt_steps[gap_mask])

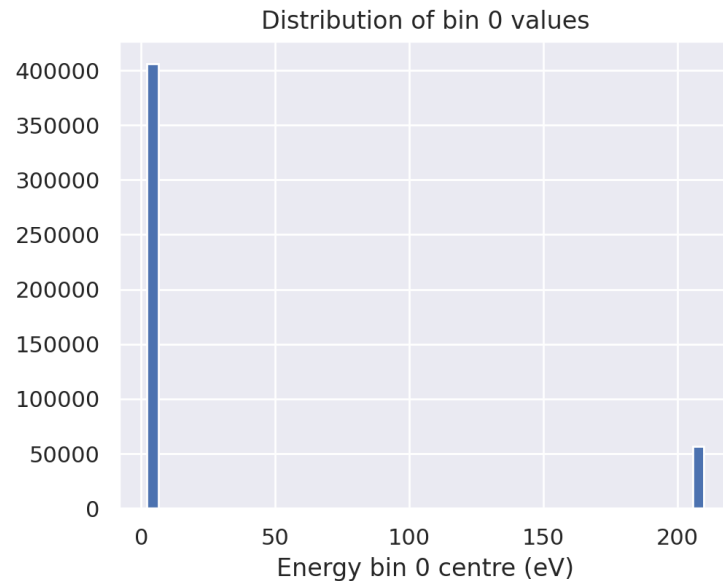
print(f"\nGaps detected: {gap_mask.sum()}")
```

Inspecting the data

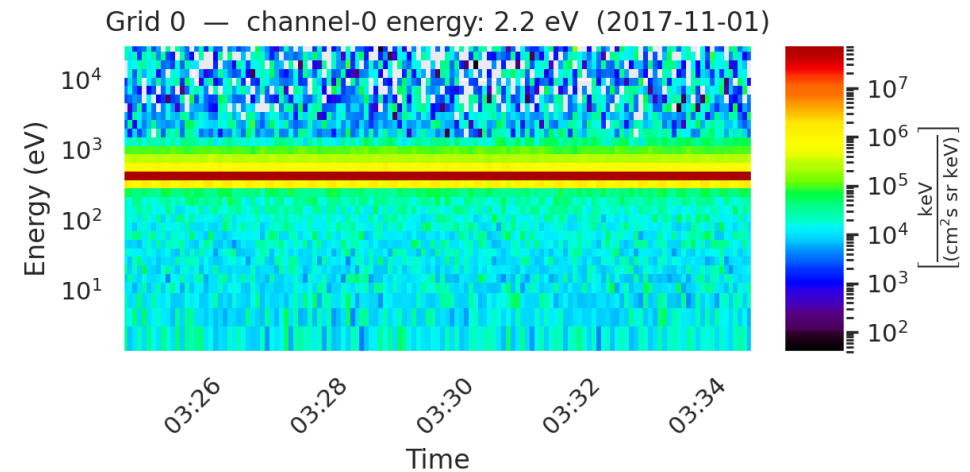
Our data:

- Omni-directional Ion Spectrum:
 - 2D, Time and Energy bins.
 - From one instrument
- Energy bin centers stored as a data artifact.
 - Let's inspect Bin 0:

Found 2 unique grids: [2.16 210.] eV (channel 0)



10-minutes of consecutive samples:



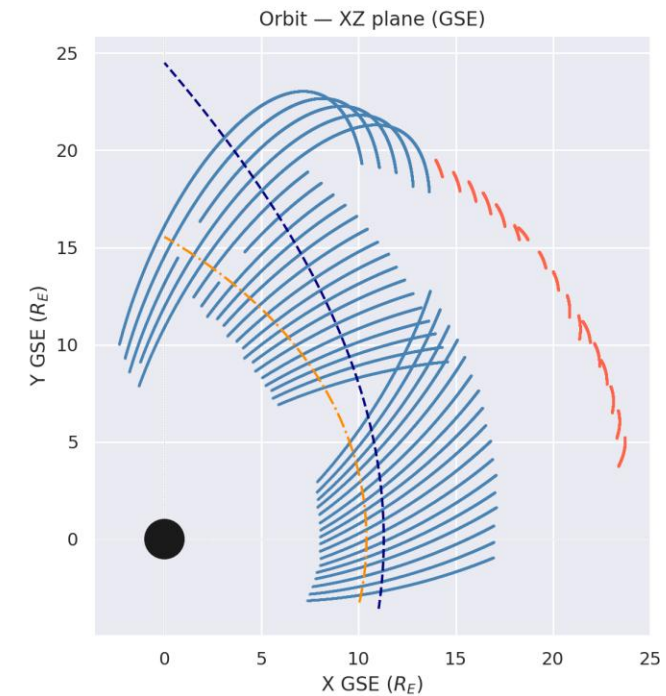
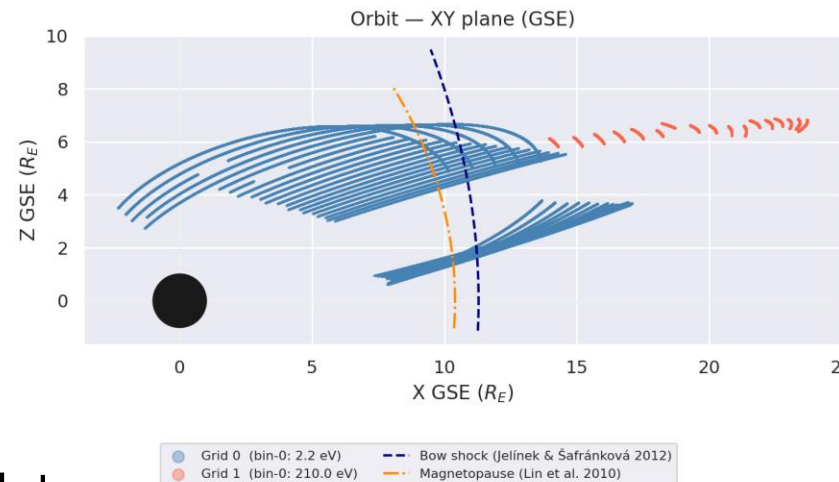
Inspecting the data

Orbital Positions:

- Second grid is in the Solar Wind
- Specific Solar wind mode used
 - Started in 2017 [3]
- Most data is on the first grid

Clean-up options:

- **Option A:** Drop the second grid data
 - Loss of data
- **Option B:** Interpolate to the first grid
 - Grid 0 is larger than Grid 1
 - Introduce artifacts where the missing grid does not exist.



Option A: Dropping Grid 1

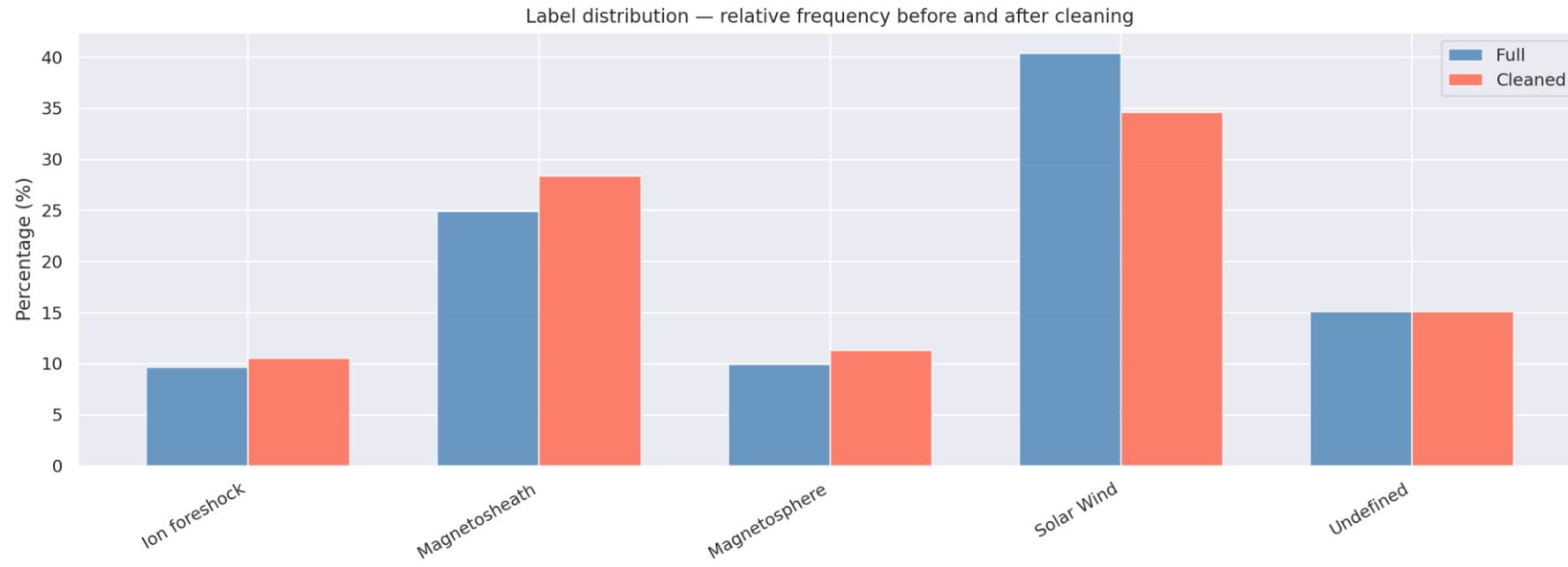
1. Use the first bin center as our target bin center.
2. Find the location of the samples using this for the first bin.
3. Mask out the rest.
4. Store you new clean dataset
5. Evaluate the distribution

```
# The dominant grid is the one whose channel-0 energy appears most often
e0 = ds["MMS1 FPI DIS energy"].isel(energy_bin=0)
target_val = float(e0.to_series().mode()[0])

# Boolean mask: True for every timestep on the dominant grid
on_target = np.isclose(e0.values, target_val, rtol=1e-3)

ds_clean = ds.isel(time=on_target)

cleaned_dataset_path = data_path / 'data_cleaned.nc'
ds_clean.to_netcdf(cleaned_dataset_path, engine='netcdf4')
```



Clean-up strategies

Time alignment:

- Resample: `ds.resample(time='1min').mean()`
- Interpolate

Value cleanup:

- Dropping data
 - Dropping specific samples
 - Drop nan samples: `ds.dropna(dim='time')`
 - Dropping an entire feature
- Interpolate: `ds.interpolate_na(dim='time')`
 - If you have data around
- Forward/backward fill: `ds.ffill(dim="time")` / `ds.bfill(dim="time")`
 - Take the last/next good data and use it
- Fill with constant: `ds.fillna(0.0)`

What is a feature?

Any measurable quantity you hand to your model as input.

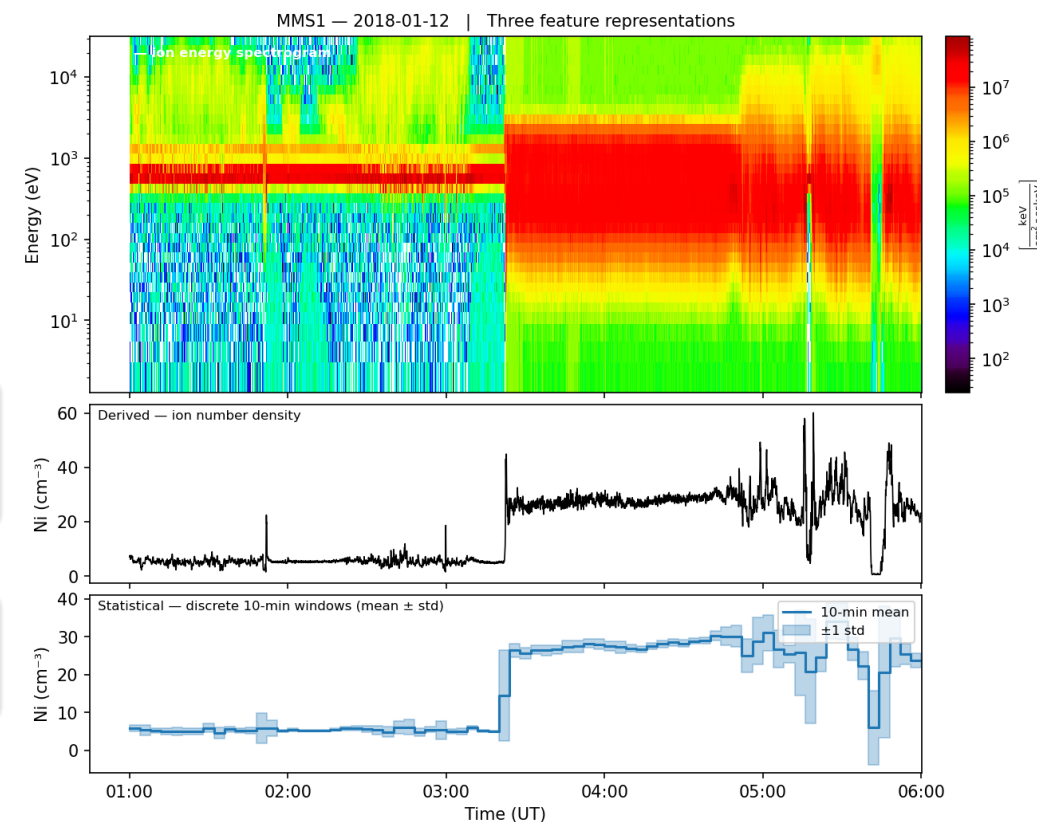
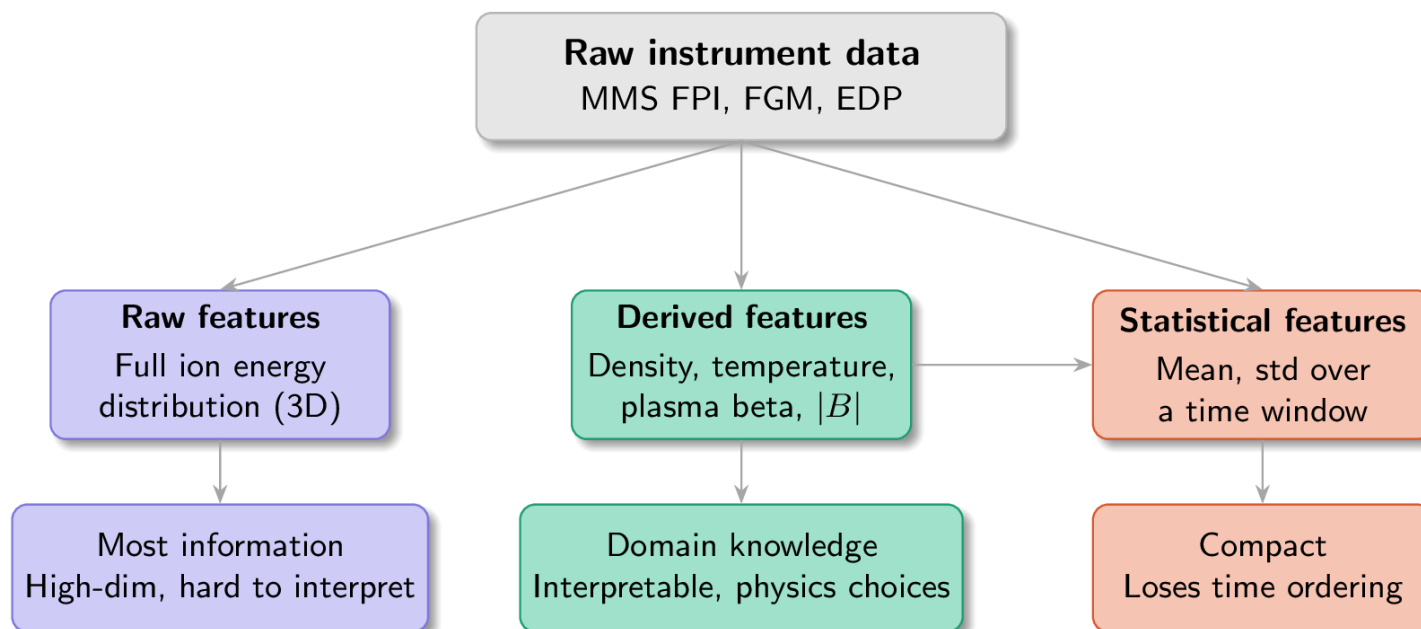
- Raw measurements, derived quantities, statistical quantities, time series

The choice of features is a physics decision, not a statistics decision

- *Physics-informed ML starts at the data level.*

You're encoding physical intuition before the model sees a single number.

Three types of feature representation:



Key Takeaways — Datasets & Data Access

Getting the data

- The magnetosphere is a dynamic environment
 - From where and when is your data?
- Access via pyspedas, cdasws, SpacePhyML, or other
 - Each with different trade-offs

General rules

- Understand your data
- Feature choice is a physics decision
- Curated datasets encode choices; read the original papers

Further reading:

xarray documentation: <https://docs.xarray.dev/en/stable/>

Pandas documentation: <https://pandas.pydata.org/docs/>

Step 0 — Understand your problem & data sources

Define the question; identify relevant variables and spatial/temporal scope

Step 1 — Select data source & variables

Choose the best-suited dataset(s); pin variable names, units, and coordinate frames

Step 2 — Download your data

Retrieve data and *cache locally*; record provenance (source, version, timestamp)

Step 3 — Inspect your data

Missing values · timeline gaps ·
time alignment between variables
Data location in time and space · physical sanity checks

Step 4 — Clean your data

Drop corrupted records · *resample* to a common cadence
Inter-/extrapolate gaps · never silently ignore anomalies

Step 5 — Store your data

Save in a portable, self-describing format (NetCDF, HDF5, Parquet, ...)
Embed metadata and version alongside processing code